gatectr.io · Developer Documentation v2.0 · Confidential
© 2025 GateCtr

© 2025 GateCtr

00 Prérequis, Outils & Premier Git

Installation depuis zéro — macOS/Linux/WSL2

Suivez cet ordre exact. Chaque outil est requis avant de passer à la phase suivante.

Outils système requis

Outil	Version	Installation	Vérification
Node.js	20 LTS	nvm install 20	`node -v`
pnpm	8.x+	npm i -g pnpm	`pnpm -v`
Git	2.40+	brew/apt install git	`git --version`
Docker Desktop	24+	hub.docker.com	`docker -v`
Docker Compose	v2.x	inclus Docker Desktop	`docker compose version`
Stripe CLI	latest	brew install stripe/stripe-cli/stripe	`stripe --version`
VS Code/Cursor	latest	cursor.sh	`code --version`

Initialisation du projet

```
pnpm create next-app@latest gatectr \
  --typescript --tailwind --eslint --app --src-dir --import-alias "@/*"
cd gatectr

# Core dependencies
pnpm add @clerk/nextjs @prisma/client zod axios nanoid bcryptjs date-fns
pnpm add @stripe/stripe-js stripe
pnpm add resend @react-email/components @react-email/render
pnpm add bullmq ioredis
pnpm add @tanstack/react-query @tanstack/react-table zustand recharts
pnpm add next-themes clsx tailwind-merge class-variance-authority
pnpm add @radix-ui/react-dialog @radix-ui/react-dropdown-menu @radix-ui/react-select
pnpm add @radix-ui/react-tabs @radix-ui/react-avatar @radix-ui/react-progress
pnpm add @sentry/nextjs

# Dev dependencies
pnpm add -D prisma vitest @vitest/coverage-v8 @testing-library/react
pnpm add -D @testing-library/jest-dom jsdom playwright @playwright/test
pnpm add -D husky lint-staged prettier eslint-config-prettier @types/bcryptjs tsx
```

Premier Git

```
git init && git branch -M main
# Ajouter au .gitignore :
echo ".env.local\n.env.*.local\n.DS_Store\n/coverage\nplaywright-report" >> .gitignore
git add . && git commit -m "chore: init GateCtr Next.js 14 project"
git remote add origin https://github.com/your-org/gatectr.git
git push -u origin main
git checkout -b develop && git push -u origin develop

# Convention Conventional Commits :
# feat(scope): description   → nouvelle feature
# fix(scope):  description   → bug fix
# chore:       description   → tooling/config
# docs:        description   → documentation
# test:        description   → tests
# ci:          description   → CI/CD
```


© 2025 GateCtr

01 Docker, Structure & Variables d'env

docker-compose complet · Dockerfile · .env

Structure des dossiers

```
gatectr/
■■■ src/
■   ■■■ app/
■   ■   ■■■ (marketing)/            # Landing, waitlist (public)
■   ■   ■■■ (auth)/                 # Clerk sign-in/sign-up
■   ■   ■■■ (dashboard)/            # App utilisateur
■   ■   ■   ■■■ layout.tsx
■   ■   ■   ■■■ page.tsx            # Overview
■   ■   ■   ■■■ analytics/          # Token analytics
■   ■   ■   ■■■ projects/           # Projets LLM
■   ■   ■   ■■■ api-keys/           # Clés API
■   ■   ■   ■■■ budget/             # Budget config
■   ■   ■   ■■■ webhooks/           # Webhooks
■   ■   ■   ■■■ billing/            # Plans & Stripe
■   ■   ■■■ (admin)/                # Espace admin restreint
■   ■   ■   ■■■ layout.tsx          # Vérif permissions RBAC
■   ■   ■   ■■■ page.tsx            # Super Admin dashboard
■   ■   ■   ■■■ users/
■   ■   ■   ■■■ plans/
■   ■   ■   ■■■ feature-flags/
■   ■   ■   ■■■ audit-logs/
■   ■   ■   ■■■ system/
■   ■   ■■■ api/
■   ■       ■■■ v1/                 # SDK public API
■   ■       ■   ■■■ complete/
■   ■       ■   ■■■ chat/
■   ■       ■   ■■■ usage/
■   ■       ■   ■■■ budget/
■   ■       ■   ■■■ webhooks/
■   ■       ■   ■■■ logs/
■   ■       ■   ■■■ models/
■   ■       ■■■ webhooks/
■   ■           ■■■ clerk/          # Sync Clerk → DB
■   ■           ■■■ stripe/         # Stripe events
■   ■■■ components/
■   ■   ■■■ ui/                     # shadcn/ui
■   ■   ■■■ dashboard/
■   ■   ■■■ admin/
■   ■   ■■■ charts/
■   ■   ■■■ emails/                 # React Email templates
■   ■■■ lib/
■   ■   ■■■ prisma.ts; redis.ts; stripe.ts; resend.ts
■   ■   ■■■ optimizer.ts; router.ts; firewall.ts; webhooks.ts
■   ■   ■■■ permissions.ts; api-auth.ts; audit.ts; features.ts
■   ■   ■■■ llm/ (openai.ts, anthropic.ts, mistral.ts)
■   ■■■ middleware.ts
■■■ prisma/ schema.prisma + migrations/
■■■ workers/ webhook.worker.ts, email.worker.ts
■■■ sdk/ src/ (SDK package)
■■■ tests/ unit/ integration/ e2e/
■■■ docker/ Dockerfile Dockerfile.dev nginx.conf
■■■ .github/workflows/ ci.yml deploy.yml
```

docker-compose.yml

© 2025 GateCtr

```
version: '3.9'
services:
  app:
    build: { context: ., dockerfile: docker/Dockerfile.dev }
    container_name: gatectr_app
    ports: ["3000:3000"]
    env_file: [.env.local]
    environment:
      DATABASE_URL: postgresql://gatectr:secret@postgres:5432/gatectr_dev
      REDIS_URL: redis://redis:6379
    volumes: [".:/app","/app/node_modules","/app/.next"]
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_healthy }
    networks: [gatectr_net]

  postgres:
    image: postgres:16-alpine; container_name: gatectr_postgres
    environment: { POSTGRES_USER: gatectr, POSTGRES_PASSWORD: secret, POSTGRES_DB: gatectr_dev }
    ports: ["5432:5432"]; volumes: [postgres_data:/var/lib/postgresql/data]
    healthcheck: { test: ["CMD-SHELL","pg_isready -U gatectr"],interval: 10s,retries: 5 }
    networks: [gatectr_net]

  redis:
    image: redis:7-alpine; container_name: gatectr_redis
    command: redis-server --appendonly yes --maxmemory 512mb
    ports: ["6379:6379"]; volumes: [redis_data:/data]
    healthcheck: { test: ["CMD","redis-cli","ping"],interval: 10s,retries: 5 }
    networks: [gatectr_net]

  worker:
    build: { context: ., dockerfile: docker/Dockerfile.dev }
    container_name: gatectr_worker
    command: npx tsx workers/webhook.worker.ts
    env_file: [.env.local]
    environment: { DATABASE_URL: ..., REDIS_URL: redis://redis:6379 }
    depends_on: [redis, postgres]; networks: [gatectr_net]

  stripe_cli:       # Stripe webhooks en dev
    image: stripe/stripe-cli:latest
    command: listen --forward-to app:3000/api/webhooks/stripe
    environment: { STRIPE_API_KEY: "${STRIPE_SECRET_KEY}" }
    networks: [gatectr_net]; profiles: [tools]

  pgadmin:
    image: dpage/pgadmin4; ports: ["5050:80"]
    environment: { PGADMIN_DEFAULT_EMAIL: admin@gatectr.io, PGADMIN_DEFAULT_PASSWORD: admin123 }
    networks: [gatectr_net]; profiles: [tools]

volumes: { postgres_data: {}, redis_data: {} }
networks: { gatectr_net: { driver: bridge } }
```

docker/Dockerfile.dev

```
FROM node:20-alpine
RUN apk add --no-cache libc6-compat && npm install -g pnpm@latest
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN pnpm install --frozen-lockfile
COPY . .
RUN npx prisma generate
EXPOSE 3000
CMD ["pnpm","dev"]
```


© 2025 GateCtr

docker/Dockerfile — Production multi-stage

```
FROM node:20-alpine AS base
RUN npm install -g pnpm@latest

FROM base AS deps
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN pnpm install --frozen-lockfile --prod

FROM base AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
ENV NEXT_TELEMETRY_DISABLED=1
RUN npx prisma generate && pnpm build

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production NEXT_TELEMETRY_DISABLED=1
RUN addgroup --system --gid 1001 nodejs && adduser --system --uid 1001 nextjs
COPY --from=builder /app/public ./public
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./.next/static
COPY --from=builder /app/prisma ./prisma
USER nextjs; EXPOSE 3000
CMD ["node","server.js"]
```

.env.example — Variables complètes

```
# APP
NEXT_PUBLIC_APP_URL="http://localhost:3000"
GATECTR_INTERNAL_SECRET="openssl rand -hex 32"

# DATABASE
DATABASE_URL="postgresql://gatectr:secret@localhost:5432/gatectr_dev"

# REDIS
REDIS_URL="redis://localhost:6379"

# CLERK AUTH
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY="pk_test_..."
CLERK_SECRET_KEY="sk_test_..."
CLERK_WEBHOOK_SECRET="whsec_..."
NEXT_PUBLIC_CLERK_SIGN_IN_URL="/sign-in"
NEXT_PUBLIC_CLERK_SIGN_UP_URL="/sign-up"
NEXT_PUBLIC_CLERK_AFTER_SIGN_IN_URL="/dashboard"

# STRIPE
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY="pk_test_..."
STRIPE_SECRET_KEY="sk_test_..."
STRIPE_WEBHOOK_SECRET="whsec_..."
STRIPE_PRICE_PRO_MONTHLY="price_..."
STRIPE_PRICE_TEAM_MONTHLY="price_..."
STRIPE_PRICE_ENTERPRISE_MONTHLY="price_..."

# RESEND
RESEND_API_KEY="re_..."
RESEND_FROM_EMAIL="noreply@gatectr.io"

# ENCRYPTION / SECURITY
ENCRYPTION_KEY="openssl rand -hex 32"
```


© 2025 GateCtr

```
WEBHOOK_SIGNING_SECRET="openssl rand -hex 32"

# MONITORING
SENTRY_DSN="https://xxx@sentry.io/xxx"
SENTRY_AUTH_TOKEN="sntrys_xxx"

# FEATURE FLAGS
ENABLE_WAITLIST="true"
ENABLE_SIGNUPS="false"
```


© 2025 GateCtr

02 Schéma DB Complet

PostgreSQL 16 · Prisma 5 · 23 tables · Relations & indexes

Toutes les tables du domaine GateCtr. Les migrations Prisma sont versionnées et appliquées via CI/CD.

Vue d'ensemble des tables

Table	Catégorie	Description
users	Core	Utilisateurs synchronisés depuis Clerk
api_keys	Security	Clés API avec hash bcrypt + prefix
waitlist_entries	Marketing	Inscrits liste d'attente pré-lancement
plans	Billing	Plans définis (FREE/PRO/TEAM/ENTERPRISE)
plan_limits	Billing	Quotas par plan (tokens, req, projets...)
subscriptions	Billing	Abonnements Stripe actifs par user
stripe_events	Billing	Idempotence events Stripe
projects	Core	Projets LLM par utilisateur
budgets	Core	Limites tokens/coût par user ou projet
usage_logs	Analytics	Log complet de chaque requête LLM
daily_usage_cache	Analytics	Agrégat quotidien pour perf
webhooks	Notifications	Endpoints configurés par user
webhook_deliveries	Notifications	Historique livraisons + retry
roles	RBAC	Rôles système (SUPER_ADMIN à VIEWER)
permissions	RBAC	Permissions granulaires par ressource
user_roles	RBAC	Assignation rôles → users
role_permissions	RBAC	Permissions → rôles
feature_flags	System	Feature flags on/off + rollout %
feature_flag_overrides	System	Override par user spécifique
audit_logs	Security	Log immuable de toutes les actions
email_logs	Email	Historique emails Resend + statuts
system_health	Monitoring	Métriques santé services (cron)
rate_limits	Security	Fenêtres rate limiting par user/IP

prisma/schema.prisma — Schéma intégral

```
generator client {
  provider = "prisma-client-js"
}
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// ■■■ AUTH & USERS ■■■
```

```
  id            String    @id @default(cuid())
  clerkId       String    @unique
  email         String    @unique
  name          String?
  avatarUrl     String?
  plan          PlanType  @default(FREE)
  planExpiresAt DateTime?
  isActive      Boolean   @default(true)
  isBanned      Boolean   @default(false)
  bannedReason  String?
  metadata      Json?     @default("{}")
  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt
  apiKeys       ApiKey[];  projects      Project[]
  webhooks      Webhook[]; usageLogs     UsageLog[]
  dailyUsage    DailyUsageCache[]; budget Budget?
  subscription  Subscription?; emailLogs EmailLog[]
  auditLogs     AuditLog[]; userRoles  UserRole[]
  featureFlagOverrides FeatureFlagOverride[]; rateLimits RateLimit[]
  @@index([clerkId]); @@index([email]); @@index([plan]); @@map("users")
}

model ApiKey {
  id           String    @id @default(cuid())
  userId       String;   name         String
  keyHash      String    @unique        // bcrypt hash of full key
  prefix       String                   // First 12 chars: "gct_xxxxxx"
  scopes       String[]  @default(["complete","read"])
  projectId    String?
  lastUsedAt   DateTime?; lastUsedIp  String?
  expiresAt    DateTime?; isActive    Boolean @default(true)
  usageCount   Int       @default(0)
  createdAt    DateTime  @default(now()); updatedAt DateTime @updatedAt
  user         User      @relation(fields:[userId],references:[id],onDelete:Cascade)
  project      Project?  @relation(fields:[projectId],references:[id])
  usageLogs    UsageLog[]
  @@index([userId]); @@index([prefix]); @@map("api_keys")
}

// ■■■ WAITLIST ■■■
```

```
  isActive             Boolean  @default(true); isPublic Boolean @default(true)
  sortOrder            Int      @default(0)
  createdAt            DateTime @default(now()); updatedAt DateTime @updatedAt
  limits               PlanLimit?; subscriptions Subscription[]
  @@map("plans")
}

model PlanLimit {
  id                      String   @id @default(cuid())
  planId                  String   @unique
  maxTokensPerDay         Int?;    maxTokensPerMonth       Int?
  maxRequestsPerDay       Int?;    maxRequestsPerMinute    Int @default(60)
  maxProjects             Int      @default(1)
  maxApiKeys              Int      @default(1);  maxWebhooks Int @default(0)
  maxTeamMembers          Int      @default(1)
  contextOptimizerEnabled Boolean  @default(false)
  modelRouterEnabled      Boolean  @default(false)
  advancedAnalytics       Boolean  @default(false)
  auditLogsRetentionDays  Int      @default(7)
  supportLevel            String   @default("community")
  createdAt               DateTime @default(now()); updatedAt DateTime @updatedAt
  plan Plan @relation(fields:[planId],references:[id])
  @@map("plan_limits")
}

model Subscription {
  id                   String    @id @default(cuid())
  userId               String    @unique
  planId               String
  stripeCustomerId     String?   @unique
  stripeSubscriptionId String?   @unique
  stripePriceId        String?
  status               SubStatus @default(INCOMPLETE)
  currentPeriodStart   DateTime?; currentPeriodEnd DateTime?
  cancelAtPeriodEnd    Boolean   @default(false); canceledAt DateTime?
  trialStart           DateTime?; trialEnd DateTime?
  createdAt            DateTime  @default(now()); updatedAt DateTime @updatedAt
  user User @relation(fields:[userId],references:[id],onDelete:Cascade)
  plan Plan @relation(fields:[planId],references:[id])
  @@index([stripeCustomerId]); @@index([stripeSubscriptionId]); @@map("subscriptions")
}

model StripeEvent {
  id        String   @id    // Stripe event ID → idempotency
  type      String;  processed Boolean @default(false)
  payload   Json;    error   String?
  createdAt DateTime @default(now())
  @@map("stripe_events")
}

enum SubStatus { INCOMPLETE INCOMPLETE_EXPIRED TRIALING ACTIVE PAST_DUE CANCELED UNPAID PAUSED }
enum PlanType  { FREE PRO TEAM ENTERPRISE }

// ■■■ PROJECTS & BUDGET ■■■
```

```
  @@index([userId]); @@index([slug]); @@map("projects")
}

model Budget {
  id                 String   @id @default(cuid())
  userId             String?  @unique; projectId String? @unique
  maxTokensPerDay    Int?;    maxTokensPerMonth Int?
  maxCostPerDay      Float?;  maxCostPerMonth   Float?
  alertThresholdPct  Int      @default(80)
  hardStop           Boolean  @default(false)
  notifyOnThreshold  Boolean  @default(true)
  notifyOnExceeded   Boolean  @default(true)
  createdAt          DateTime @default(now()); updatedAt DateTime @updatedAt
  user    User?    @relation(fields:[userId],references:[id])
  project Project? @relation(fields:[projectId],references:[id])
  @@map("budgets")
}

// ■■■ USAGE & ANALYTICS ■■■
```

```
  webhookId  String;  event String; payload Json
  status     Int;     responseMs Int; success Boolean
  error      String?; retryCount Int @default(0)
  createdAt  DateTime @default(now())
  webhook Webhook @relation(fields:[webhookId],references:[id],onDelete:Cascade)
  @@index([webhookId,createdAt]); @@map("webhook_deliveries")
}

// ■■■ RBAC ■■■
```

```
  ipAddress  String?; userAgent String?
  success    Boolean  @default(true); error String?
  createdAt  DateTime @default(now())
  user User? @relation(fields:[userId],references:[id])
  @@index([userId,createdAt]); @@index([resource,action]); @@index([createdAt]); @@map("audit_logs")
}

// ■■■ EMAIL ■■■
```

PHASE 0

Waitlist & Landing Pré-lancement

Page d'attente · Inscription · Table waitlist_entries · Admin view

0

03 Phase 0 — Waitlist

Landing pré-lancement avec gestion des inscrits

Avant d'ouvrir les inscriptions, déployez la page waitlist. Activez le flag ENABLE_WAITLIST=true. Cette phase collecte les emails, calcule le rang et invite par batch.

Page Waitlist — src/app/(marketing)/waitlist/page.tsx

```
// Page publique de la waitlist
'use client';
import { useState } from 'react';

export default function WaitlistPage() {
  const [email,setEmail]=useState(''); const [name,setName]=useState('');
  const [useCase,setUseCase]=useState(''); const [submitted,setSubmitted]=useState(false);
  const [position,setPosition]=useState<number|null>(null);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    const res = await fetch('/api/waitlist', {
      method:'POST',
      headers:{'Content-Type':'application/json'},
      body: JSON.stringify({ email, name, useCase }),
    });
    const data = await res.json();
    if (res.ok) { setSubmitted(true); setPosition(data.position); }
  };

  if (submitted) return (
    <div className="min-h-screen flex items-center justify-center bg-[#060810]">
      <div className="text-center space-y-4 max-w-md px-6">
        <div className="text-5xl">■</div>
        <h2 className="text-2xl font-bold text-white">You're on the list!</h2>
        <p className="text-blue-300">Position #{position} — We'll notify you at {email}</p>
        <p className="text-slate-400 text-sm">Share your referral link to move up the list.</p>
      </div>
    </div>
  );

  return (
    <div className="min-h-screen bg-[#060810] flex items-center justify-center">
      <div className="max-w-lg w-full px-6 space-y-8">
        <div className="text-center space-y-3">
          <h1 className="text-4xl font-bold text-white">GateCtr</h1>
          <p className="text-blue-300 text-lg">The universal LLM middleware</p>
          <p className="text-slate-400">Join the waitlist — early access coming soon.</p>
        </div>
        <form onSubmit={handleSubmit} className="space-y-4">
          <input type="text" placeholder="Your name" value={name}
            onChange={e=>setName(e.target.value)}
            className="w-full bg-slate-800 border border-slate-700 rounded-lg px-4 py-3 text-white"/>
          <input type="email" placeholder="your@email.com" required value={email}
            onChange={e=>setEmail(e.target.value)}
            className="w-full bg-slate-800 border border-slate-700 rounded-lg px-4 py-3 text-white"/>
          <select value={useCase} onChange={e=>setUseCase(e.target.value)}
            className="w-full bg-slate-800 border border-slate-700 rounded-lg px-4 py-3 text-white">
```


© 2025 GateCtr

```
            <option value="">I'm building...</option>
            <option value="saas">A SaaS product</option>
            <option value="agent">AI agents / automation</option>
            <option value="enterprise">Enterprise internal tools</option>
            <option value="dev">Personal / dev projects</option>
          </select>
          <button type="submit"
            className="w-full bg-[#00B4C8] hover:bg-[#00D4E8] text-white font-bold py-3 rounded-lg transition">
            Join the Waitlist →
          </button>
        </form>
      </div>
    </div>
  );
}
```

API Waitlist — src/app/api/waitlist/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { z } from 'zod';
import { prisma } from '@/lib/prisma';
import { sendWelcomeWaitlistEmail } from '@/lib/resend';

const Schema = z.object({
  email: z.string().email(),
  name: z.string().optional(),
  useCase: z.string().optional(),
  referredBy: z.string().optional(),
});

export async function POST(req: NextRequest) {
  const body = await req.json();
  const parsed = Schema.safeParse(body);
  if (!parsed.success) return NextResponse.json({ error: 'Invalid input' }, { status: 400 });

  const { email, name, useCase, referredBy } = parsed.data;

  // Check if already registered
  const existing = await prisma.waitlistEntry.findUnique({ where: { email } });
  if (existing) return NextResponse.json({ position: existing.position, alreadyRegistered: true });

  const entry = await prisma.waitlistEntry.create({
    data: {
      email, name, useCase, referredBy,
      ipAddress: req.headers.get('x-forwarded-for') ?? req.headers.get('x-real-ip') ?? 'unknown',
      userAgent: req.headers.get('user-agent') ?? '',
    },
  });

  // Send welcome email async
  sendWelcomeWaitlistEmail({ to: email, name, position: entry.position }).catch(console.error);

  return NextResponse.json({ position: entry.position, success: true });
}

export async function GET() {
  const count = await prisma.waitlistEntry.count({ where: { status: 'WAITING' } });
  return NextResponse.json({ waitingCount: count });
}
```

Admin Waitlist — Vue Super Admin

© 2025 GateCtr

```
// src/app/(admin)/waitlist/page.tsx
import { requirePermission } from '@/lib/permissions';
import { prisma } from '@/lib/prisma';

export default async function AdminWaitlistPage() {
  await requirePermission('users', 'read');

  const entries = await prisma.waitlistEntry.findMany({
    orderBy: { position: 'asc' },
    take: 100,
  });

  const stats = {
    total:   await prisma.waitlistEntry.count(),
    waiting: await prisma.waitlistEntry.count({ where: { status: 'WAITING' } }),
    invited: await prisma.waitlistEntry.count({ where: { status: 'INVITED' } }),
    joined:  await prisma.waitlistEntry.count({ where: { status: 'JOINED'  } }),
  };

  return (
    <div className="space-y-6">
      <div className="flex justify-between items-center">
        <h1 className="text-2xl font-bold">Waitlist Management</h1>
        <InviteBatchButton />   {/* Invites next N users */}
      </div>
      <StatsRow stats={stats} />
      <WaitlistTable entries={entries} />
    </div>
  );
}

// Action: inviter un batch d'utilisateurs
async function inviteBatch(count: number) {
  const toInvite = await prisma.waitlistEntry.findMany({
    where: { status: 'WAITING' },
    orderBy: { position: 'asc' },
    take: count,
  });
  for (const entry of toInvite) {
    await prisma.waitlistEntry.update({
      where: { id: entry.id },
      data: { status: 'INVITED', invitedAt: new Date() },
    });
    await sendInviteEmail({ to: entry.email, name: entry.name });
  }
}
```

i Feature flag ENABLE_WAITLIST=true active la redirection vers /waitlist. ENABLE_SIGNUPS=false bloque les inscriptions directes via Clerk.

© 2025 GateCtr

PHASE 1

Auth Clerk & RBAC Complet

Middleware · Sync users · Rôles · Permissions granulaires

1

04 Phase 1 — Auth Clerk & RBAC

Middleware · User sync · Permissions · Interfaces par rôle

Middleware d'authentification

```
// src/middleware.ts
import { clerkMiddleware, createRouteMatcher } from '@clerk/nextjs/server';
import { NextResponse } from 'next/server';

const isPublic = createRouteMatcher([
  '/', '/waitlist', '/api/waitlist(.*)',
  '/sign-in(.*)', '/sign-up(.*)',
  '/api/v1/(.*)',      // API publique SDK (auth via API key)
  '/api/health',
]);
const isAdmin = createRouteMatcher(['/admin(.*)']);

export default clerkMiddleware(async (auth, req) => {
  // Vérifier le feature flag waitlist
  if (!isPublic(req)) {
    const { userId } = auth();
    if (!userId) return NextResponse.redirect(new URL('/sign-in', req.url));
  }
  if (isAdmin(req)) {
    const { userId } = auth();
    if (!userId) return NextResponse.redirect(new URL('/sign-in', req.url));
    // La vérification du rôle admin se fait dans le layout
  }
});

export const config = {
  matcher: ['/((?!.*\..*|_next).*)', '/', '/(api|trpc)(.*)'],
};
```

Webhook Clerk → Sync DB

```
// src/app/api/webhooks/clerk/route.ts
import { Webhook } from 'svix';
import { headers } from 'next/headers';
import { prisma } from '@/lib/prisma';
import { sendWelcomeEmail } from '@/lib/resend';
import { auditLog } from '@/lib/audit';

export async function POST(req: Request) {
  const payload = await req.text();
  const heads = Object.fromEntries(headers());
  const wh = new Webhook(process.env.CLERK_WEBHOOK_SECRET!);

  let evt: any;
  try { evt = wh.verify(payload, heads); }
  catch { return new Response('Invalid signature', { status: 400 }); }

  switch (evt.type) {
    case 'user.created': {
      const { id, email_addresses, first_name, last_name, image_url } = evt.data;
      const email = email_addresses[0]?.email_address;
```


© 2025 GateCtr

```
      const user = await prisma.user.create({
        data: {
          clerkId: id, email,
          name: `${first_name ?? ''} ${last_name ?? ''}`.trim(),
          avatarUrl: image_url,
        },
      });
      // Assign default role DEVELOPER
      const devRole = await prisma.role.findUnique({ where: { name: 'DEVELOPER' } });
      if (devRole) {
        await prisma.userRole.create({ data: { userId: user.id, roleId: devRole.id } });
      }
      // Send welcome email
      await sendWelcomeEmail({ to: email, name: user.name });
      await auditLog({ resource: 'user', action: 'create', resourceId: user.id, newValue: { email } });
      break;
    }
    case 'user.updated': {
      const { id, email_addresses, first_name, last_name } = evt.data;
      await prisma.user.update({
        where: { clerkId: id },
        data: {
          email: email_addresses[0]?.email_address,
          name: `${first_name ?? ''} ${last_name ?? ''}`.trim(),
        },
      });
      break;
    }
    case 'user.deleted': {
      const { id } = evt.data;
      await prisma.user.update({ where: { clerkId: id }, data: { isActive: false } });
      break;
    }
  }
  return new Response('OK', { status: 200 });
}
```

Système de Permissions RBAC

```
// src/lib/permissions.ts
import { auth } from '@clerk/nextjs/server';
import { prisma } from './prisma';
import { cache } from './redis';
import { redirect } from 'next/navigation';

// Matrix de permissions par rôle
const ROLE_PERMISSIONS: Record<string, string[]> = {
  SUPER_ADMIN: ['*:*'],   // Tout
  ADMIN:       ['users:read','users:write','users:ban','analytics:read',
                'analytics:export','billing:read','system:read','audit:read'],
  MANAGER:     ['analytics:read','users:read','billing:read'],
  DEVELOPER:   ['analytics:read'],
  VIEWER:      ['analytics:read'],
  SUPPORT:     ['users:read','audit:read'],
};

export async function getUserPermissions(userId: string): Promise<string[]> {
  const cacheKey = `perms:${userId}`;
  const cached = await cache.get(cacheKey);
  if (cached) return JSON.parse(cached);

  const userRoles = await prisma.userRole.findMany({
```


© 2025 GateCtr

```
    where: { userId },
    include: { role: true },
  });

  const permissions = new Set<string>();
  for (const { role } of userRoles) {
    const perms = ROLE_PERMISSIONS[role.name] ?? [];
    perms.forEach(p => permissions.add(p));
  }

  const result = Array.from(permissions);
  await cache.set(cacheKey, JSON.stringify(result), 300); // Cache 5 min
  return result;
}

export async function hasPermission(resource: string, action: string): Promise<boolean> {
  const { userId: clerkId } = auth();
  if (!clerkId) return false;
  const user = await prisma.user.findUnique({ where: { clerkId } });
  if (!user) return false;
  const perms = await getUserPermissions(user.id);
  return perms.includes('*:*') || perms.includes(`${resource}:${action}`);
}

// Server component guard — redirige si pas autorisé
export async function requirePermission(resource: string, action: string) {
  const allowed = await hasPermission(resource, action);
  if (!allowed) redirect('/dashboard?error=unauthorized');
}

// Client hook
export async function getUserRole(userId: string): Promise<string> {
  const userRole = await prisma.userRole.findFirst({
    where: { userId }, include: { role: true },
    orderBy: { createdAt: 'desc' },
  });
  return userRole?.role.name ?? 'VIEWER';
}
```

RBAC — Matrice des accès

Ressource / Action	SUPER_ADMIN	ADMIN	MANAGER	DEVELOPER	VIEWER	SUPPORT
users: read	■	■	■	—	—	■
users: write	■	■	—	—	—	—
users: delete	■	—	—	—	—	—
users: ban	■	■	—	—	—	—
analytics: read	■	■	■	■	■	—
analytics: export	■	■	■	—	—	—
billing: read	■	■	■	—	—	—
billing: manage	■	—	—	—	—	—
system: admin	■	—	—	—	—	—
system: read	■	■	—	—	—	—
features: manage	■	—	—	—	—	—

© 2025 GateCtr

audit: read	■	■	—	—	—	■
webhooks: manage	■	■	—	—	—	—

Admin Layout — Protection par rôle

```
// src/app/(admin)/layout.tsx
import { auth } from '@clerk/nextjs/server';
import { redirect } from 'next/navigation';
import { prisma } from '@/lib/prisma';
import { getUserPermissions } from '@/lib/permissions';
import { AdminSidebar } from '@/components/admin/AdminSidebar';

export default async function AdminLayout({ children }) {
  const { userId: clerkId } = auth();
  if (!clerkId) redirect('/sign-in');

  const user = await prisma.user.findUnique({
    where: { clerkId },
    include: { userRoles: { include: { role: true } } },
  });

  const roleNames = user?.userRoles.map(ur => ur.role.name) ?? [];
  const isAdmin = roleNames.some(r =>
    ['SUPER_ADMIN','ADMIN','MANAGER','SUPPORT'].includes(r)
  );
  if (!isAdmin) redirect('/dashboard?error=unauthorized');

  const permissions = await getUserPermissions(user!.id);
  const highestRole = roleNames.includes('SUPER_ADMIN') ? 'SUPER_ADMIN'
    : roleNames.includes('ADMIN') ? 'ADMIN'
    : roleNames.includes('MANAGER') ? 'MANAGER' : 'SUPPORT';

  return (
    <div className="flex h-screen bg-slate-950">
      <AdminSidebar role={highestRole} permissions={permissions} />
      <main className="flex-1 overflow-y-auto">{children}</main>
    </div>
  );
}
```


© 2025 GateCtr

PHASE 2

Stripe & Gestion des Plans

Checkout · Webhooks · Contrôle quotas · Portail facturation

2

05 Phase 2 — Stripe & Plans

Checkout · Webhooks Stripe · Quota enforcement

Stripe Créer les produits et prix dans le Stripe Dashboard avant de coder. Copier les price_xxx dans .env.

lib/stripe.ts — Client Stripe

```
// src/lib/stripe.ts
import Stripe from 'stripe';

export const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: '2024-06-20',
  typescript: true,
});

export const PLANS = {
  PRO: {
    priceId: process.env.STRIPE_PRICE_PRO_MONTHLY!,
    name: 'Pro',
    amount: 2900,  // €29/mo in cents
  },
  TEAM: {
    priceId: process.env.STRIPE_PRICE_TEAM_MONTHLY!,
    name: 'Team',
    amount: 9900,  // €99/mo
  },
  ENTERPRISE: {
    priceId: process.env.STRIPE_PRICE_ENTERPRISE_MONTHLY!,
    name: 'Enterprise',
    amount: null,   // Custom
  },
} as const;

// Créer ou récupérer un customer Stripe
export async function getOrCreateStripeCustomer(userId: string, email: string) {
  const { prisma } = await import('./prisma');
  const sub = await prisma.subscription.findUnique({ where: { userId } });
  if (sub?.stripeCustomerId) return sub.stripeCustomerId;

  const customer = await stripe.customers.create({
    email,
    metadata: { userId },
  });

  await prisma.subscription.upsert({
    where: { userId },
    update: { stripeCustomerId: customer.id },
    create: {
      userId, planId: (await prisma.plan.findUnique({ where: { name: 'FREE' } }))!.id,
      stripeCustomerId: customer.id, status: 'INCOMPLETE',
    },
  });
  return customer.id;
}
```


© 2025 GateCtr

Checkout Session — Upgrade vers plan payant

```
// src/app/api/billing/checkout/route.ts
import { auth } from '@clerk/nextjs/server';
import { NextRequest, NextResponse } from 'next/server';
import { stripe, getOrCreateStripeCustomer, PLANS } from '@/lib/stripe';
import { prisma } from '@/lib/prisma';
import { z } from 'zod';

const Schema = z.object({ plan: z.enum(['PRO','TEAM','ENTERPRISE']) });

export async function POST(req: NextRequest) {
  const { userId: clerkId } = auth();
  if (!clerkId) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

  const user = await prisma.user.findUnique({ where: { clerkId } });
  if (!user) return NextResponse.json({ error: 'User not found' }, { status: 404 });

  const { plan } = Schema.parse(await req.json());
  const planConfig = PLANS[plan];

  const customerId = await getOrCreateStripeCustomer(user.id, user.email);

  const session = await stripe.checkout.sessions.create({
    customer: customerId,
    payment_method_types: ['card'],
    line_items: [{ price: planConfig.priceId, quantity: 1 }],
    mode: 'subscription',
    success_url: `${process.env.NEXT_PUBLIC_APP_URL}/billing?success=true&session_id={CHECKOUT_SESSION_ID}`,
    cancel_url:  `${process.env.NEXT_PUBLIC_APP_URL}/billing?canceled=true`,
    metadata: { userId: user.id, plan },
    subscription_data: {
      trial_period_days: 14,  // 14 jours d'essai gratuit
      metadata: { userId: user.id },
    },
  });

  return NextResponse.json({ url: session.url });
}
```

Webhook Stripe — Events Handler

```
// src/app/api/webhooks/stripe/route.ts
import { NextRequest } from 'next/server';
import { stripe } from '@/lib/stripe';
import { prisma } from '@/lib/prisma';
import { auditLog } from '@/lib/audit';

export async function POST(req: NextRequest) {
  const payload = await req.text();
  const sig = req.headers.get('stripe-signature')!;

  let event: any;
  try { event = stripe.webhooks.constructEvent(payload, sig, process.env.STRIPE_WEBHOOK_SECRET!); }
  catch { return new Response('Webhook Error', { status: 400 }); }

  // Idempotency — éviter le double traitement
  const existing = await prisma.stripeEvent.findUnique({ where: { id: event.id } });
  if (existing?.processed) return new Response('Already processed', { status: 200 });

  await prisma.stripeEvent.upsert({
    where: { id: event.id },
    update: {},
```


© 2025 GateCtr

```
    create: { id: event.id, type: event.type, payload: event as any },
  });

  try {
    switch (event.type) {
      case 'customer.subscription.created':
      case 'customer.subscription.updated': {
        const sub = event.data.object;
        const userId = sub.metadata?.userId;
        if (!userId) break;
        const planName = getPlanFromPriceId(sub.items.data[0].price.id);
        await prisma.$transaction([
          prisma.user.update({ where: { id: userId }, data: { plan: planName as any } }),
          prisma.subscription.update({
            where: { userId },
            data: {
              stripeSubscriptionId: sub.id,
              stripePriceId: sub.items.data[0].price.id,
              status: sub.status.toUpperCase() as any,
              currentPeriodStart: new Date(sub.current_period_start * 1000),
              currentPeriodEnd:   new Date(sub.current_period_end   * 1000),
              cancelAtPeriodEnd:  sub.cancel_at_period_end,
            },
          }),
        ]);
        await auditLog({ resource:'billing', action:'subscription_updated',
          resourceId: userId, newValue: { plan: planName, status: sub.status } });
        break;
      }
      case 'customer.subscription.deleted': {
        const sub = event.data.object;
        const userId = sub.metadata?.userId;
        if (!userId) break;
        await prisma.$transaction([
          prisma.user.update({ where: { id: userId }, data: { plan: 'FREE' } }),
          prisma.subscription.update({
            where: { userId },
            data: { status: 'CANCELED', canceledAt: new Date() },
          }),
        ]);
        break;
      }
      case 'invoice.payment_failed': {
        const invoice = event.data.object;
        const customerId = invoice.customer;
        const sub = await prisma.subscription.findUnique({ where: { stripeCustomerId: customerId } });
        if (sub) {
          await prisma.subscription.update({ where: { id: sub.id }, data: { status: 'PAST_DUE' } });
          // Send payment failed email
          const user = await prisma.user.findUnique({ where: { id: sub.userId } });
          if (user) await sendPaymentFailedEmail({ to: user.email, name: user.name });
        }
        break;
      }
    }
    await prisma.stripeEvent.update({ where: { id: event.id }, data: { processed: true } });
  } catch (err: any) {
    await prisma.stripeEvent.update({ where: { id: event.id }, data: { error: err.message } });
    return new Response('Handler error', { status: 500 });
  }
  return new Response('OK', { status: 200 });
}
```


© 2025 GateCtr

```
function getPlanFromPriceId(priceId: string): string {
  if (priceId === process.env.STRIPE_PRICE_PRO_MONTHLY)   return 'PRO';
  if (priceId === process.env.STRIPE_PRICE_TEAM_MONTHLY)  return 'TEAM';
  if (priceId === process.env.STRIPE_PRICE_ENTERPRISE_MONTHLY) return 'ENTERPRISE';
  return 'FREE';
}
```

Quota Enforcement — Vérification Plan

```
// src/lib/plan-guard.ts
import { prisma } from './prisma';
import { cache } from './redis';

export async function checkPlanLimit(
  userId: string,
  check: 'tokens_day' | 'requests_minute' | 'projects' | 'api_keys' | 'webhooks'
) {
  const cacheKey = `plan:${userId}`;
  let limits: any = null;

  const cached = await cache.get(cacheKey);
  if (cached) { limits = JSON.parse(cached); }
  else {
    const user = await prisma.user.findUnique({
      where: { id: userId },
      include: { subscription: { include: { plan: { include: { limits: true } } } } } },
    });
    limits = user?.subscription?.plan.limits;
    if (limits) await cache.set(cacheKey, JSON.stringify(limits), 300);
  }

  if (!limits) return { allowed: true };  // FREE defaults apply

  switch (check) {
    case 'projects': {
      if (limits.maxProjects === -1) return { allowed: true };
      const count = await prisma.project.count({ where: { userId, isActive: true } });
      if (count >= limits.maxProjects) return { allowed: false,
        reason: `Plan limit: max ${limits.maxProjects} projects`,
        upgrade: true };
      return { allowed: true, remaining: limits.maxProjects - count };
    }
    case 'api_keys': {
      if (limits.maxApiKeys === -1) return { allowed: true };
      const count = await prisma.apiKey.count({ where: { userId, isActive: true } });
      return count >= limits.maxApiKeys
        ? { allowed: false, reason: `Plan limit: max ${limits.maxApiKeys} API keys`, upgrade: true }
        : { allowed: true };
    }
    case 'webhooks': {
      if (limits.maxWebhooks === 0) return { allowed: false,
        reason: 'Webhooks not available on Free plan', upgrade: true };
      if (limits.maxWebhooks === -1) return { allowed: true };
      const count = await prisma.webhook.count({ where: { userId, isActive: true } });
      return count >= limits.maxWebhooks
        ? { allowed: false, reason: `Plan limit: max ${limits.maxWebhooks} webhooks`, upgrade: true }
        : { allowed: true };
    }
    default: return { allowed: true };
  }
}
```


© 2025 GateCtr

Plans & Quotas — Tableau de référence

Quota	FREE	PRO	TEAM	ENTERPRISE
Tokens/jour	50K	2M	10M	Illimité
Tokens/mois	500K	20M	100M	Illimité
Req/min	10	60	200	1 000
Projets	1	5	Illimité	Illimité
Clés API	1	5	20	Illimité
Webhooks	0	10	50	Illimité
Team members	1	1	20	Illimité
Context Optimizer	—	■	■	■
Model Router	—	■	■	■
Analytics avancées	—	■	■	■
Audit logs	7 jours	30 jours	90 jours	365 jours
Support	Community	Email	Prioritaire	Dédié
Tarif mensuel	€0	€29	€99	Custom

© 2025 GateCtr

PHASE 3

Core API & Budget Firewall

Auth API Key · /complete · /chat · Budget control

3

06 Phase 3 — Core API

API Key auth · /complete · /chat · Budget Firewall · Rate limiting

Génération de clés API

```
// src/app/api/v1/keys/route.ts
import { auth } from '@clerk/nextjs/server';
import { NextRequest, NextResponse } from 'next/server';
import bcrypt from 'bcryptjs';
import { nanoid } from 'nanoid';
import { prisma } from '@/lib/prisma';
import { checkPlanLimit } from '@/lib/plan-guard';
import { auditLog } from '@/lib/audit';

export async function POST(req: NextRequest) {
  const { userId: clerkId } = auth();
  if (!clerkId) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  const user = await prisma.user.findUnique({ where: { clerkId } });
  if (!user) return NextResponse.json({ error: 'Not found' }, { status: 404 });

  // Check plan limit
  const check = await checkPlanLimit(user.id, 'api_keys');
  if (!check.allowed) return NextResponse.json({ error: check.reason, upgrade: check.upgrade }, { status: 403
});

  const { name, scopes, projectId, expiresAt } = await req.json();

  // Generate key: "gct_" + 32 random chars
  const rawKey = `gct_${nanoid(32)}`;
  const prefix = rawKey.substring(0, 12);
  const keyHash = await bcrypt.hash(rawKey, 12);

  const apiKey = await prisma.apiKey.create({
    data: { userId: user.id, name, keyHash, prefix, scopes, projectId, expiresAt },
  });

  await auditLog({ userId: user.id, resource: 'api_key', action: 'create',
    resourceId: apiKey.id, newValue: { name, scopes } });

  // Return the raw key ONCE — never stored in plain text
  return NextResponse.json({ id: apiKey.id, key: rawKey, prefix, name,
    warning: 'Save this key now. It will not be shown again.' });
}
```

Auth Middleware API Key

```
// src/lib/api-auth.ts
import { NextRequest } from 'next/server';
import bcrypt from 'bcryptjs';
import { prisma } from './prisma';
import { checkRateLimit } from './rate-limit';

export async function authenticateApiKey(req: NextRequest) {
  const auth = req.headers.get('authorization');
  if (!auth?.startsWith('Bearer gct_'))
    return { error: 'Missing or invalid Authorization header', status: 401 };
```


© 2025 GateCtr

```
  const rawKey = auth.replace('Bearer ', '');
  const prefix = rawKey.substring(0, 12);
  const ip = req.headers.get('x-forwarded-for') ?? 'unknown';

  // Rate limit check (before DB hit)
  const rl = await checkRateLimit(`ip:${ip}`, 200, 60);
  if (!rl.allowed) return { error: 'Rate limit exceeded', status: 429, retryAfter: rl.reset };

  // Fetch by prefix (fast — no full scan)
  const keys = await prisma.apiKey.findMany({
    where: { prefix, isActive: true, OR: [{ expiresAt: null }, { expiresAt: { gt: new Date() } }] },
    include: { user: { include: { subscription: { include: { plan: { include: { limits: true } } } } } } },
  });

  for (const key of keys) {
    const valid = await bcrypt.compare(rawKey, key.keyHash);
    if (valid) {
      if (key.user.isBanned) return { error: 'Account suspended', status: 403 };
      // Update usage async
      prisma.apiKey.update({ where: { id: key.id },
        data: { lastUsedAt: new Date(), lastUsedIp: ip, usageCount: { increment: 1 } }
      }).catch(console.error);
      return { apiKey: key, user: key.user };
    }
  }
  return { error: 'Invalid API key', status: 401 };
}
```

POST /api/v1/complete — Endpoint principal

```
// src/app/api/v1/complete/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { z } from 'zod';
import { authenticateApiKey } from '@/lib/api-auth';
import { checkBudget, updateBudgetUsage } from '@/lib/firewall';
import { optimizeContext } from '@/lib/optimizer';
import { routeModel } from '@/lib/router';
import { callLLM } from '@/lib/llm';
import { checkPlanLimit } from '@/lib/plan-guard';
import { checkRateLimit } from '@/lib/rate-limit';
import { logUsage } from '@/lib/analytics';

const Schema = z.object({
  model:       z.string().default('auto'),
  messages:    z.array(z.object({ role: z.enum(['system','user','assistant']), content: z.string() })),
  projectId:   z.string().optional(),
  optimize:    z.boolean().default(true),
  stream:      z.boolean().default(false),
  maxTokens:   z.number().max(100000).optional(),
  temperature: z.number().min(0).max(2).optional(),
});

export async function POST(req: NextRequest) {
  // 1. Auth
  const auth = await authenticateApiKey(req);
  if ('error' in auth && auth.error) return NextResponse.json({ error: auth.error }, { status: auth.status });
  const { apiKey, user } = auth as any;

  // 2. Per-minute rate limit by user
  const limits = user.subscription?.plan.limits;
  const maxReqPerMin = limits?.maxRequestsPerMinute ?? 10;
```


© 2025 GateCtr

```
  const rl = await checkRateLimit(`user:${user.id}:rpm`, maxReqPerMin, 60);
  if (!rl.allowed) return NextResponse.json({ error: 'Rate limit exceeded', retry_after: rl.reset }, { status:
429 });

  // 3. Parse & validate body
  const parsed = Schema.safeParse(await req.json());
  if (!parsed.success) return NextResponse.json({ error: parsed.error.flatten() }, { status: 400 });
  const input = parsed.data;

  // 4. Plan feature check
  if (input.optimize && !limits?.contextOptimizerEnabled) {
    input.optimize = false;  // silently disable if plan doesn't include it
  }

  // 5. Budget check
  const budget = await checkBudget(user.id, input.projectId);
  if (!budget.allowed) return NextResponse.json({ error: 'Budget limit exceeded',
    details: budget.reason, current: budget.current }, { status: 429 });

  // 6. Optimize context
  const { messages, savedTokens } = input.optimize
    ? await optimizeContext(input.messages)
    : { messages: input.messages, savedTokens: 0 };

  // 7. Route model
  const { model, provider, llmKey } = await routeModel(input.model, user, messages);

  // 8. Call LLM
  const t0 = Date.now();
  let response: any;
  try { response = await callLLM({ model, provider, llmKey, messages, maxTokens: input.maxTokens, temperature:
input.temperature }); }
  catch (err: any) {
    await logUsage({ userId: user.id, projectId: input.projectId, apiKeyId: apiKey.id,
      model, provider, promptTokens: 0, completionTokens: 0, totalTokens: 0,
      savedTokens: 0, costUsd: 0, latencyMs: Date.now()-t0, statusCode: 500,
      errorCode: err.code, errorMessage: err.message });
    return NextResponse.json({ error: 'LLM call failed', code: err.code }, { status: 502 });
  }
  const latency = Date.now() - t0;

  // 9. Update budget cache + log async
  updateBudgetUsage(user.id, input.projectId, response.usage.total_tokens).catch(console.error);
  logUsage({ userId: user.id, projectId: input.projectId, apiKeyId: apiKey.id,
    model, provider, promptTokens: response.usage.prompt_tokens,
    completionTokens: response.usage.completion_tokens,
    totalTokens: response.usage.total_tokens, savedTokens,
    costUsd: response.cost, latencyMs: latency,
    optimized: input.optimize, routed: input.model === 'auto' }).catch(console.error);

  return NextResponse.json({
    content: response.content, model, provider,
    usage: { prompt_tokens: response.usage.prompt_tokens,
      completion_tokens: response.usage.completion_tokens,
      total_tokens: response.usage.total_tokens,
      saved_tokens: savedTokens, cost_usd: response.cost },
    latency_ms: latency,
  });
}
```

Rate Limiting Redis — Sliding Window

© 2025 GateCtr

```
// src/lib/rate-limit.ts
import { cache } from './redis';

export async function checkRateLimit(key: string, limit: number, windowSec: number) {
  const now = Math.floor(Date.now() / 1000);
  const windowKey = `rl:${key}:${Math.floor(now / windowSec)}`;
  const current = await cache.incr(windowKey);
  if (current === 1) await cache.set(windowKey, String(current), windowSec + 1);
  return {
    allowed: current <= limit,
    remaining: Math.max(0, limit - current),
    reset: (Math.floor(now / windowSec) + 1) * windowSec,
  };
}
```


© 2025 GateCtr

PHASE 4

Modules Métier

Context Optimizer · Model Router · Webhook Engine

4

07

Phase 4 — Modules

Optimizer · Router · Webhook Engine · LLM adapters

Context Optimizer

```
// src/lib/optimizer.ts
export async function optimizeContext(messages: any[], opts = { keepLastN: 8, maxChars: 16000 }) {
  const original = messages.reduce((a, m) => a + Math.ceil(m.content.length / 4), 0);
  // 1. Keep system + last N messages
  const sys = messages.filter(m => m.role === 'system');
  const convo = messages.filter(m => m.role !== 'system').slice(-opts.keepLastN);
  // 2. Compress system prompt whitespace
  const compressed = [...sys.map(m => ({ ...m, content: m.content.replace(/\s+/g,' ').trim() })), ...convo];
  // 3. Truncate very long messages
  const truncated = compressed.map(m =>
    m.content.length > opts.maxChars
      ? { ...m, content: m.content.slice(0, opts.maxChars) + '...[truncated]' }
      : m
  );
  const after = truncated.reduce((a, m) => a + Math.ceil(m.content.length / 4), 0);
  return { messages: truncated, savedTokens: Math.max(0, original - after), ratio: after / original };
}
```

Model Router & LLM Adapters

```
// src/lib/router.ts
const MODELS = [
  { id:'gpt-4o-mini',     provider:'openai',    cost:0.00015, quality:0.82, ctx:128000 },
  { id:'gpt-4o',          provider:'openai',    cost:0.005,   quality:0.95, ctx:128000 },
  { id:'claude-haiku',    provider:'anthropic', cost:0.00025, quality:0.80, ctx:200000 },
  { id:'claude-sonnet',   provider:'anthropic', cost:0.003,   quality:0.92, ctx:200000 },
  { id:'mistral-small',   provider:'mistral',   cost:0.0002,  quality:0.78, ctx:32000  },
  { id:'gemini-flash',    provider:'google',    cost:0.000075,quality:0.79, ctx:1000000},
];

export async function routeModel(requested: string, user: any, messages: any[], strategy='balanced') {
  if (requested !== 'auto') return resolveExplicitModel(requested, user);
  const estTokens = messages.reduce((a, m) => a + Math.ceil(m.content.length/4), 0);
  const isPro = ['PRO','TEAM','ENTERPRISE'].includes(user.plan);
  const eligible = MODELS.filter(m => {
    if (!isPro && m.cost > 0.001) return false;  // FREE: cheaper models only
    return m.ctx >= estTokens;
  });
  const scored = eligible.map(m => ({
    ...m, score: strategy==='cost' ? -m.cost : strategy==='performance' ? m.quality
      : m.quality*0.6 - m.cost*100*0.4
  })).sort((a,b) => b.score - a.score);
  const sel = scored[0];
  const llmKey = getUserProviderKey(user, sel.provider);
  return { model: sel.id, provider: sel.provider, llmKey };
}

// src/lib/llm/index.ts — Unified adapter
export async function callLLM({ model, provider, llmKey, messages, maxTokens, temperature }: any) {
  switch (provider) {
    case 'openai':    return callOpenAI({ model, apiKey: llmKey, messages, maxTokens, temperature });
```


© 2025 GateCtr

```
    case 'anthropic': return callAnthropic({ model, apiKey: llmKey, messages, maxTokens, temperature });
    case 'mistral':   return callMistral({ model, apiKey: llmKey, messages, maxTokens, temperature });
    case 'google':    return callGemini({ model, apiKey: llmKey, messages, maxTokens, temperature });
    default: throw new Error(`Unknown provider: ${provider}`);
  }
}
```

Webhook Engine — BullMQ

```
// src/lib/webhooks.ts
import { Queue } from 'bullmq';
import { redis } from './redis';

export const webhookQueue = new Queue('webhooks', {
  connection: redis,
  defaultJobOptions: { attempts: 3, backoff: { type: 'exponential', delay: 2000 },
    removeOnComplete: 100, removeOnFail: 500 },
});

export async function triggerWebhook(userId: string, event: string, payload: any) {
  await webhookQueue.add('dispatch', { userId, event, payload, timestamp: new Date().toISOString() });
}

// workers/webhook.worker.ts
import { Worker } from 'bullmq';
import crypto from 'crypto';
import axios from 'axios';
import { prisma } from '../src/lib/prisma';

new Worker('webhooks', async (job) => {
  const { userId, event, payload, timestamp } = job.data;
  const hooks = await prisma.webhook.findMany({ where: { userId, isActive: true, events: { has: event } } });
  for (const wh of hooks) {
    const body = JSON.stringify({ event, timestamp, data: payload });
    const sig  = crypto.createHmac('sha256', wh.secret).update(body).digest('hex');
    const t0   = Date.now();
    try {
      const res = await axios.post(wh.url, body, {
        headers: { 'Content-Type':'application/json',
          'X-GateCtr-Signature': `sha256=${sig}`, 'X-GateCtr-Event': event },
        timeout: 10000,
      });
      await prisma.webhookDelivery.create({ data: { webhookId:wh.id, event, payload,
        status:res.status, responseMs:Date.now()-t0, success:true } });
      await prisma.webhook.update({ where:{id:wh.id}, data:{successCount:{increment:1},lastFiredAt:new Date()}
});
    } catch(err: any) {
      await prisma.webhookDelivery.create({ data: { webhookId:wh.id, event, payload,
        status:err?.response?.status??0, responseMs:Date.now()-t0, success:false,
        error:err.message?.slice(0,255) } });
      await prisma.webhook.update({ where:{id:wh.id}, data:{failCount:{increment:1}} });
    }
  }
}, { connection: redis, concurrency: 10 });
```

Événements webhook disponibles

Événement	Déclencheur	Payload
`budget.exceeded`	Budget dépassé (hard stop)	user_id, tokens_used, limit, project_id
`token.threshold`	Seuil d'alerte atteint (ex: 80%)	percentage, tokens_remaining, reset_date

© 2025 GateCtr

`request.failed`	Erreur LLM provider (timeout, 5xx)	error_code, model, retry_count
`model.switched`	Fallback automatique modèle	original_model, new_model, reason
`plan.upgraded`	Abonnement mis à jour	old_plan, new_plan, user_id
`api_key.created`	Nouvelle clé API créée	key_id, name, scopes
`usage.daily`	Rapport quotidien (minuit)	total_tokens, cost, top_projects
`security.alert`	Tentative injection détectée	input_hash, rule_triggered, severity

© 2025 GateCtr

PHASE 5

Resend & Système de Mails

Templates React Email · Email queue · Logs

5

08 Phase 5 — Resend & Emails

React Email templates · Email queue · Logging

Configuration Resend

```
// src/lib/resend.ts
import { Resend } from 'resend';
import { prisma } from './prisma';

export const resend = new Resend(process.env.RESEND_API_KEY);
const FROM = process.env.RESEND_FROM_EMAIL ?? 'noreply@gatectr.io';

type EmailOptions = { to: string; userId?: string; subject: string;
  template: string; html: string; text?: string; };

export async function sendEmail({ to, userId, subject, template, html, text }: EmailOptions) {
  try {
    const { data, error } = await resend.emails.send({
      from: `GateCtr <${FROM}>`, to, subject, html,
      text: text ?? html.replace(/<[^>]*>/g, ''),
    });
    // Log to DB
    await prisma.emailLog.create({ data: {
      userId, to, subject, template,
      resendId: data?.id, status: error ? 'FAILED' : 'SENT',
      error: error?.message,
    }});
    return { success: !error, id: data?.id };
  } catch (err: any) {
    await prisma.emailLog.create({ data: { userId, to, subject, template,
      status: 'FAILED', error: err.message } });
    return { success: false, error: err.message };
  }
}

// ■■ EMAIL FUNCTIONS ■■■
```

```
  { to:string; name?:string|null; percentage:number; tokensRemaining:number }) {
  const html = render(BudgetAlertEmail({ name: name ?? 'User', percentage, tokensRemaining }));
  return sendEmail({ to, subject: `■■ GateCtr: ${percentage}% of your daily token budget used`,
template:'budget_alert', html });
}

export async function sendPaymentFailedEmail({ to, name }:
  { to:string; name?:string|null }) {
  const html = render(PaymentFailedEmail({ name: name ?? 'User' }));
  return sendEmail({ to, subject: '■■ Payment issue with your GateCtr subscription',
template:'payment_failed', html });
}
```

React Email Template — Welcome

```
// src/components/emails/WelcomeEmail.tsx
import { Html, Head, Body, Container, Section, Heading,
  Text, Button, Link, Img } from '@react-email/components';

export function WelcomeEmail({ name }: { name: string }) {
  return (
    <Html>
      <Head />
      <Body style={{ backgroundColor:'#060810', fontFamily:'Helvetica' }}>
        <Container style={{ maxWidth:'600px', margin:'0 auto', padding:'40px 20px' }}>
          <Section style={{ textAlign:'center', marginBottom:'32px' }}>
            <Img src="https://gatectr.io/logo-email.png" width="120" height="40" alt="GateCtr"/>
          </Section>
          <Section style={{ backgroundColor:'#0D1117', borderRadius:'12px',
            border:'1px solid #30363D', padding:'32px' }}>
            <Heading style={{ color:'#FFFFFF', fontSize:'24px', fontWeight:'bold' }}>
              Welcome to GateCtr, {name}! ■
            </Heading>
            <Text style={{ color:'#8B949E', lineHeight:'1.6' }}>
              Your LLM middleware is ready. Start optimizing your requests in minutes.
            </Text>
            <Button href="https://app.gatectr.io/dashboard"
              style={{ backgroundColor:'#00B4C8', color:'white', padding:'12px 28px',
                borderRadius:'6px', fontWeight:'bold', textDecoration:'none', display:'block',
                textAlign:'center', marginTop:'24px' }}>
              Go to Dashboard →
            </Button>
          </Section>
          <Text style={{ color:'#484F58', fontSize:'12px', textAlign:'center', marginTop:'24px' }}>
            GateCtr · <Link href="https://gatectr.io" style={{ color:'#00B4C8' }}>gatectr.io</Link>
          </Text>
        </Container>
      </Body>
    </Html>
  );
}
```

Templates email disponibles

Template	Déclencheur	Sujet
`welcome`	Inscription Clerk → webhook user.created	Welcome to GateCtr!
`waitlist_confirm`	POST /api/waitlist	You're #N on the waitlist
`waitlist_invite`	Admin invite batch	You're invited — Claim your spot

© 2025 GateCtr

`budget_alert`	Seuil budget atteint (80%)	■■ X% of your token budget used
`budget_exceeded`	Dépassement budget (hard stop)	Budget limit reached — upgrade now
`payment_failed`	Stripe invoice.payment_failed	Payment issue — action required
`subscription_started`	Stripe subscription created	Your Pro plan is active!
`subscription_canceled`	Stripe subscription deleted	Your subscription has ended
`api_key_created`	POST /api/v1/keys	New API key created
`security_alert`	Injection détectée	Security alert on your account

© 2025 GateCtr

PHASE 6

Admin Space & Super Admin

Dashboard admin · Gestion users · Analytics globales · Audit

6

09 Phase 6 — Admin Space

Super Admin · Admin views · User management · Audit logs

Super Admin Dashboard

```
// src/app/(admin)/page.tsx — Super Admin overview
import { requirePermission } from '@/lib/permissions';
import { prisma } from '@/lib/prisma';

export default async function SuperAdminPage() {
  await requirePermission('system', 'read');

  const [totalUsers, activeToday, totalTokens, revenue] = await Promise.all([
    prisma.user.count(),
    prisma.usageLog.groupBy({ by: ['userId'],
      where: { createdAt: { gte: new Date(Date.now() - 86400000) } } }).then(r => r.length),
    prisma.usageLog.aggregate({ _sum: { totalTokens: true } }).then(r => r._sum.totalTokens ?? 0),
    prisma.subscription.count({ where: { status: 'ACTIVE' } }),
  ]);

  const planBreakdown = await prisma.user.groupBy({
    by: ['plan'], _count: { id: true }
  });

  return (
    <div className="space-y-6 p-6">
      <div className="flex items-center justify-between">
        <h1 className="text-2xl font-bold text-white">Super Admin</h1>
        <Badge variant="destructive">SUPER_ADMIN</Badge>
      </div>

      {/* Stats cards */}
      <div className="grid grid-cols-4 gap-4">
        <StatCard label="Total Users"     value={totalUsers}   icon="■" color="blue"/>
        <StatCard label="Active Today"    value={activeToday}  icon="■" color="green"/>
        <StatCard label="Total Tokens"    value={totalTokens}  icon="■" color="cyan"/>
        <StatCard label="Active Sub"      value={revenue}      icon="■" color="amber"/>
      </div>

      {/* Plan breakdown */}
      <PlanBreakdownChart data={planBreakdown} />

      {/* Recent audit logs */}
      <RecentAuditLogs limit={20} />

      {/* System health */}
      <SystemHealthPanel />
    </div>
  );
}
```

Admin Sidebar — Navigation par rôle

```
// src/components/admin/AdminSidebar.tsx
'use client';
import { usePathname } from 'next/navigation';
```


© 2025 GateCtr

```
import Link from 'next/link';

const NAV_ITEMS = [
  { href:'/admin',              label:'Dashboard',     icon:'■', perm:'system:read' },
  { href:'/admin/users',        label:'Users',         icon:'■', perm:'users:read'  },
  { href:'/admin/waitlist',     label:'Waitlist',      icon:'■', perm:'users:read'  },
  { href:'/admin/plans',        label:'Plans & Billing',icon:'■', perm:'billing:read'},
  { href:'/admin/analytics',    label:'Analytics',     icon:'■', perm:'analytics:read'},
  { href:'/admin/feature-flags',label:'Feature Flags', icon:'■', perm:'features:manage'},
  { href:'/admin/audit-logs',   label:'Audit Logs',    icon:'■', perm:'audit:read'  },
  { href:'/admin/webhooks',     label:'Webhooks',      icon:'■', perm:'webhooks:manage'},
  { href:'/admin/system',       label:'System Health', icon:'■', perm:'system:admin'},
];

export function AdminSidebar({ role, permissions }: { role: string; permissions: string[] }) {
  const pathname = usePathname();
  const isSuperAdmin = permissions.includes('*:*');

  const visible = NAV_ITEMS.filter(item => {
    if (isSuperAdmin) return true;
    const [resource, action] = item.perm.split(':');
    return permissions.includes(`${resource}:${action}`);
  });

  return (
    <aside className="w-56 bg-slate-950 border-r border-slate-800 flex flex-col">
      <div className="p-4 border-b border-slate-800">
        <span className="text-white font-bold">GateCtr Admin</span>
        <Badge className="ml-2" variant="outline">{role}</Badge>
      </div>
      <nav className="flex-1 p-2 space-y-1">
        {visible.map(item => (
          <Link key={item.href} href={item.href}
            className={`flex items-center gap-2 px-3 py-2 rounded-lg text-sm transition
              ${pathname === item.href
                ? 'bg-cyan-500/10 text-cyan-400 font-medium'
                : 'text-slate-400 hover:text-white hover:bg-slate-800'}`}>
            <span>{item.icon}</span>
            {item.label}
          </Link>
        ))}
      </nav>
    </aside>
  );
}
```

Gestion des utilisateurs — Admin

```
// src/app/(admin)/users/page.tsx
import { requirePermission } from '@/lib/permissions';
import { prisma } from '@/lib/prisma';

export default async function AdminUsersPage({ searchParams }) {
  await requirePermission('users', 'read');

  const page = parseInt(searchParams?.page ?? '1');
  const query = searchParams?.q ?? '';
  const PER_PAGE = 25;

  const [users, total] = await Promise.all([
    prisma.user.findMany({
      where: query ? {
```


© 2025 GateCtr

```
        OR: [{ email: { contains: query, mode: 'insensitive' } },
             { name:  { contains: query, mode: 'insensitive' } }]
      } : {},
      include: { subscription: { include: { plan: true } }, userRoles: { include: { role: true } } },
      orderBy: { createdAt: 'desc' },
      take: PER_PAGE, skip: (page - 1) * PER_PAGE,
    }),
    prisma.user.count(),
  ]);

  return (
    <div className="p-6 space-y-4">
      <div className="flex justify-between items-center">
        <h1 className="text-xl font-bold text-white">Users ({total})</h1>
        <UserSearchBar defaultValue={query} />
      </div>
      <UserTable users={users} />
      <Pagination page={page} total={total} perPage={PER_PAGE} />
    </div>
  );
}

// Actions admin : ban, change plan, assign role
async function banUser(userId: string, reason: string) {
  await requirePermission('users', 'ban');
  await prisma.user.update({ where:{id:userId}, data:{isBanned:true, bannedReason:reason} });
  await auditLog({ resource:'user', action:'ban', resourceId:userId, newValue:{reason} });
  await sendAccountSuspendedEmail({ userId });
}

async function changePlan(userId: string, newPlan: string) {
  await requirePermission('billing', 'manage');
  await prisma.user.update({ where:{id:userId}, data:{plan:newPlan as any} });
  await auditLog({ resource:'user', action:'plan_change', resourceId:userId, newValue:{plan:newPlan} });
}
```


© 2025 GateCtr

PHASE 7

SDK & Documentation Technique

Node.js SDK · Python SDK · REST API docs · Exemples

7

10 Phase 7 — SDK & Docs

SDK Node.js · Python · REST reference · OpenAPI spec

SDK Node.js / TypeScript

```
// sdk/src/client.ts — GateCtr SDK
export class GateCtr {
  private readonly baseUrl: string;
  private readonly apiKey: string;
  private readonly options: GateCtrOptions;

  constructor(options: GateCtrOptions) {
    this.apiKey  = options.apiKey;
    this.baseUrl = options.baseUrl ?? 'https://api.gatectr.io';
    this.options = options;
  }

  async complete(params: CompleteParams): Promise<CompleteResponse> {
    const res = await fetch(`${this.baseUrl}/api/v1/complete`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${this.apiKey}`,
        'X-GateCtr-SDK': 'node/1.0.0',
      },
      body: JSON.stringify({
        model:       params.model ?? 'auto',
        messages:    params.messages,
        projectId:   params.projectId,
        optimize:    params.optimize ?? true,
        maxTokens:   params.maxTokens,
        temperature: params.temperature,
      }),
    });
    if (!res.ok) {
      const err = await res.json().catch(() => ({}));
      throw new GateCtrError(err.error ?? 'API error', res.status, err);
    }
    return res.json();
  }

  async chat(params: ChatParams): Promise<CompleteResponse> {
    return this.complete({ ...params, model: params.model ?? 'auto' });
  }

  async getUsage(params?: UsageParams): Promise<UsageResponse> {
    const qs = new URLSearchParams(params as any).toString();
    const res = await fetch(`${this.baseUrl}/api/v1/usage?${qs}`, {
      headers: { 'Authorization': `Bearer ${this.apiKey}` },
    });
    return res.json();
  }

  async getBudget(): Promise<BudgetResponse> {
    const res = await fetch(`${this.baseUrl}/api/v1/budget`, {
      headers: { 'Authorization': `Bearer ${this.apiKey}` },
```


© 2025 GateCtr

```
    });
    return res.json();
  }
}

// Usage example
const gate = new GateCtr({
  apiKey: process.env.GATECTR_API_KEY!,
  projectId: 'my-app',
});

const response = await gate.complete({
  messages: [{ role: 'user', content: 'Summarize this document...' }],
  optimize: true,
  model: 'auto',
});
console.log(response.content);
// { content: "...", usage: { total_tokens: 840, saved_tokens: 320, cost_usd: 0.00012 } }
```

SDK Python

```
# sdk/python/gatectr/client.py
import requests
from typing import Optional, List, Dict, Any

class GateCtr:
    def __init__(self, api_key: str, base_url: str = "https://api.gatectr.io",
                 project_id: Optional[str] = None):
        self.api_key    = api_key
        self.base_url   = base_url
        self.project_id = project_id
        self.session    = requests.Session()
        self.session.headers.update({
            "Authorization": f"Bearer {api_key}",
            "Content-Type": "application/json",
            "X-GateCtr-SDK": "python/1.0.0",
        })

    def complete(self, messages: List[Dict], model: str = "auto",
                 optimize: bool = True, max_tokens: Optional[int] = None,
                 project_id: Optional[str] = None) -> Dict[str, Any]:
        res = self.session.post(f"{self.base_url}/api/v1/complete", json={
            "model": model, "messages": messages,
            "projectId": project_id or self.project_id,
            "optimize": optimize, "maxTokens": max_tokens,
        })
        res.raise_for_status()
        return res.json()

    def get_usage(self, date: Optional[str] = None, project_id: Optional[str] = None):
        params = {}
        if date:       params["date"]      = date
        if project_id: params["projectId"] = project_id
        return self.session.get(f"{self.base_url}/api/v1/usage", params=params).json()

    def get_budget(self):
        return self.session.get(f"{self.base_url}/api/v1/budget").json()

# Usage
from gatectr import GateCtr
import os
```


© 2025 GateCtr

```
gate = GateCtr(api_key=os.getenv("GATECTR_API_KEY"), project_id="my-agent")
result = gate.complete(
    messages=[{"role":"user","content":"Explain quantum computing in 2 sentences."}],
    optimize=True
)
print(result["content"])
# {"content":"...","usage":{"total_tokens":180,"saved_tokens":40,"cost_usd":0.000027}}
```

REST API Reference complète

Méthode	Endpoint	Auth	Description
POST	/api/v1/complete	**API Key**	Requête LLM optimisée (principal)
POST	/api/v1/chat	**API Key**	Conversation multi-tours
GET	/api/v1/usage	**API Key**	Consommation tokens (filtres: date, project)
GET	/api/v1/budget	**API Key**	Statut budget et limites configurées
POST	/api/v1/budget	**Clerk JWT**	Configurer/mettre à jour le budget
GET	/api/v1/webhooks	**Clerk JWT**	Lister tous les webhooks
POST	/api/v1/webhooks	**Clerk JWT**	Créer un webhook endpoint
PUT	/api/v1/webhooks/:id	**Clerk JWT**	Mettre à jour un webhook
DELETE	/api/v1/webhooks/:id	**Clerk JWT**	Supprimer un webhook
GET	/api/v1/logs	**API Key**	Historique requêtes (pagination)
GET	/api/v1/models	**API Key**	Modèles disponibles + scoring
POST	/api/v1/keys	**Clerk JWT**	Créer une API key
DELETE	/api/v1/keys/:id	**Clerk JWT**	Révoquer une API key
GET	/api/health	**Public**	Health check (DB + Redis)

© 2025 GateCtr

PHASE 8

Monitoring Admin & Users

Health checks · Métriques temps réel · Alertes · Dashboards

8

11 Phase 8 — Monitoring

Health API · Admin metrics · User analytics · Alertes

Health Check Endpoint

```
// src/app/api/health/route.ts
import { NextResponse } from 'next/server';
import { prisma } from '@/lib/prisma';
import { redis } from '@/lib/redis';
import { webhookQueue } from '@/lib/webhooks';

export async function GET() {
  const t0 = Date.now();
  const checks: Record<string, any> = {
    status: 'ok', timestamp: new Date().toISOString(),
    version: process.env.npm_package_version ?? '1.0.0',
    services: {},
  };

  // Database
  try {
    const t = Date.now();
    await prisma.$queryRaw`SELECT 1`;
    checks.services.database = { status:'healthy', latencyMs: Date.now()-t };
  } catch(e: any) {
    checks.services.database = { status:'down', error: e.message };
    checks.status = 'degraded';
  }

  // Redis
  try {
    const t = Date.now();
    await redis.ping();
    checks.services.redis = { status:'healthy', latencyMs: Date.now()-t };
  } catch(e: any) {
    checks.services.redis = { status:'down', error: e.message };
    checks.status = 'degraded';
  }

  // BullMQ Queue
  try {
    const queueInfo = await webhookQueue.getJobCounts();
    checks.services.queue = { status:'healthy', jobs: queueInfo };
  } catch(e: any) {
    checks.services.queue = { status:'down', error: e.message };
  }

  checks.totalLatencyMs = Date.now() - t0;
  const status = checks.status === 'ok' ? 200 : 503;

  // Persist to DB for history
  for (const [service, info] of Object.entries(checks.services) as any[]) {
    await prisma.systemHealth.create({
      data: { service, status: info.status === 'healthy' ? 'HEALTHY' : 'DOWN',
        latencyMs: info.latencyMs, errorMessage: info.error }
    }).catch(()=>{});
```


© 2025 GateCtr

```
  }
  return NextResponse.json(checks, { status });
}
```

Admin System Health Dashboard

```
// src/app/(admin)/system/page.tsx
import { requirePermission } from '@/lib/permissions';
import { prisma } from '@/lib/prisma';

export default async function SystemPage() {
  await requirePermission('system', 'read');

  // Dernières 24h de métriques
  const since = new Date(Date.now() - 24*3600*1000);
  const healthHistory = await prisma.systemHealth.findMany({
    where: { checkedAt: { gte: since } },
    orderBy: { checkedAt: 'desc' }, take: 200,
  });

  // Métriques globales en temps réel
  const [queueStats, errorRate, avgLatency] = await Promise.all([
    getQueueStats(),
    prisma.usageLog.count({ where: { statusCode: { gte: 500 }, createdAt: { gte: since } } }),
    prisma.usageLog.aggregate({ where: { createdAt: { gte: since } }, _avg: { latencyMs: true } }),
  ]);

  return (
    <div className="p-6 space-y-6">
      <h1 className="text-xl font-bold text-white">System Health</h1>
      <ServiceStatusGrid history={healthHistory} />
      <div className="grid grid-cols-3 gap-4">
        <MetricCard label="Queue Jobs (pending)" value={queueStats.waiting} color="blue"/>
        <MetricCard label="Errors (24h)" value={errorRate} color="red"/>
        <MetricCard label="Avg Latency (ms)" value={avgLatency._avg.latencyMs?.toFixed(0)} color="green"/>
      </div>
      <HealthHistoryChart data={healthHistory} />
      <RecentErrors limit={50} />
    </div>
  );
}
```

User Analytics Dashboard

```
// src/app/(dashboard)/analytics/page.tsx
import { auth } from '@clerk/nextjs/server';
import { prisma } from '@/lib/prisma';

export default async function AnalyticsPage({ searchParams }) {
  const { userId: clerkId } = auth();
  const user = await prisma.user.findUnique({ where:{clerkId}, include:{budget:true} });

  const period = searchParams?.period ?? '7d';
  const since = getPeriodStart(period);

  const [usageByDay, usageByModel, usageByProject, topStats] = await Promise.all([
    prisma.usageLog.groupBy({ by:['createdAt'], where:{ userId:user!.id, createdAt:{gte:since} },
      _sum:{ totalTokens:true, savedTokens:true, costUsd:true }, _count:{ id:true } }),
    prisma.usageLog.groupBy({ by:['model','provider'], where:{ userId:user!.id, createdAt:{gte:since} },
      _sum:{ totalTokens:true }, _count:{ id:true }, orderBy:{ _sum:{ totalTokens:'desc' } }, take:10 }),
    prisma.usageLog.groupBy({ by:['projectId'], where:{ userId:user!.id, createdAt:{gte:since} },
      _sum:{ totalTokens:true, costUsd:true }, orderBy:{ _sum:{ totalTokens:'desc' } }, take:10 }),
```


© 2025 GateCtr

```
    prisma.usageLog.aggregate({ where:{ userId:user!.id, createdAt:{gte:since} },
      _sum:{ totalTokens:true, savedTokens:true, costUsd:true }, _count:{ id:true }, _avg:{ latencyMs:true }
}),
  ]);

  return (
    <div className="space-y-6">
      <div className="flex justify-between items-center">
        <h1 className="text-2xl font-bold">Analytics</h1>
        <PeriodSelector current={period} />
      </div>
      <TopStatsRow stats={topStats} />
      <div className="grid grid-cols-2 gap-6">
        <TokensOverTimeChart data={usageByDay} />
        <CostBreakdownChart data={usageByDay} />
      </div>
      <div className="grid grid-cols-2 gap-6">
        <ModelUsageChart data={usageByModel} />
        <ProjectUsageTable data={usageByProject} />
      </div>
      <BudgetStatusCard budget={user?.budget} usage={topStats} />
    </div>
  );
}
```


© 2025 GateCtr

PHASE 9

Feature Flags & Publication

Activation features · Rollout % · Override user · Admin UI

9

12 Phase 9 — Feature Flags

Feature flags system · Rollout · Admin publication

Feature Flags Service

```
// src/lib/features.ts
import { prisma } from './prisma';
import { cache } from './redis';

export async function isFeatureEnabled(key: string, userId?: string, plan?: string): Promise<boolean> {
  const cacheKey = `ff:${key}:${userId ?? 'global'}`;
  const cached = await cache.get(cacheKey);
  if (cached !== null) return cached === '1';

  const flag = await prisma.featureFlag.findUnique({
    where: { key },
    include: { overrides: userId ? { where: { userId } } : false },
  });
  if (!flag) { await cache.set(cacheKey,'0',60); return false; }
  if (!flag.enabled) { await cache.set(cacheKey,'0',60); return false; }

  // User-specific override
  if (userId && flag.overrides && flag.overrides.length > 0) {
    const override = flag.overrides[0];
    await cache.set(cacheKey, override.enabled ? '1':'0', 300);
    return override.enabled;
  }

  // Plan check
  if (flag.enabledPlans.length > 0 && plan) {
    const hasAccess = flag.enabledPlans.includes(plan);
    if (!hasAccess) { await cache.set(cacheKey,'0',300); return false; }
  }

  // Percentage rollout
  if (flag.rolloutPct < 100 && userId) {
    const hash = parseInt(userId.slice(-8), 16) % 100;
    const enabled = hash < flag.rolloutPct;
    await cache.set(cacheKey, enabled ? '1':'0', 300);
    return enabled;
  }

  await cache.set(cacheKey, '1', 300);
  return true;
}
```

Admin Feature Flags UI

```
// src/app/(admin)/feature-flags/page.tsx
import { requirePermission } from '@/lib/permissions';
import { prisma } from '@/lib/prisma';

export default async function FeatureFlagsPage() {
  await requirePermission('features', 'manage');
  const flags = await prisma.featureFlag.findMany({
    include: { _count: { select: { overrides: true } } },
```


© 2025 GateCtr

```
    orderBy: { createdAt: 'asc' },
  });
  return (
    <div className="p-6 space-y-4">
      <div className="flex justify-between">
        <h1 className="text-xl font-bold text-white">Feature Flags</h1>
        <CreateFlagButton />
      </div>
      <div className="space-y-3">
        {flags.map(flag => (
          <FlagCard key={flag.id} flag={flag} />
        ))}
      </div>
    </div>
  );
}

// Activer/désactiver un flag — Server Action
async function toggleFlag(flagId: string, enabled: boolean) {
  'use server';
  await requirePermission('features', 'manage');
  await prisma.featureFlag.update({ where: { id: flagId }, data: { enabled } });
  // Invalider le cache Redis
  await invalidateFeatureFlagCache(flagId);
  await auditLog({ resource: 'feature_flag', action: enabled ? 'enable':'disable', resourceId: flagId });
}

// Mettre à jour le rollout %
async function updateRollout(flagId: string, rolloutPct: number) {
  'use server';
  await requirePermission('features', 'manage');
  await prisma.featureFlag.update({ where:{id:flagId}, data:{rolloutPct} });
  await invalidateFeatureFlagCache(flagId);
}
```


© 2025 GateCtr

PHASE 10

Sécurité, Audit & Tests

Input validation · Audit logs · Tests unitaires · Pentest checklist

10

13 Phase 10 — Sécurité & Tests

Audit logs · Input security · Tests stratégie · Pentest

Audit Log Service

```
// src/lib/audit.ts
import { prisma } from './prisma';
import { auth } from '@clerk/nextjs/server';

interface AuditParams {
  userId?: string; actorId?: string; resource: string; action: string;
  resourceId?: string; oldValue?: any; newValue?: any;
  ipAddress?: string; userAgent?: string; success?: boolean; error?: string;
}

export async function auditLog(params: AuditParams) {
  try {
    await prisma.auditLog.create({
      data: {
        userId:     params.userId,
        actorId:    params.actorId,
        resource:   params.resource,
        action:     params.action,
        resourceId: params.resourceId,
        oldValue:   params.oldValue,
        newValue:   params.newValue,
        ipAddress:  params.ipAddress,
        userAgent:  params.userAgent,
        success:    params.success ?? true,
        error:      params.error,
      },
    });
  } catch (err) {
    // Audit failures should never break business logic
    console.error('[AUDIT] Failed to write audit log:', err);
  }
}
```

Tests — Stratégie complète

Type	Framework	Coverage cible	Dossier
Unit tests	Vitest	90%+	tests/unit/
Integration tests	Vitest + real DB	80%+	tests/integration/
E2E tests	Playwright	Flows critiques	tests/e2e/
API tests	REST Client / Playwright	Tous endpoints	tests/api/
Security tests	Manual + OWASP ZAP	OWASP Top 10	docs/security/
Load tests	k6	500 req/s soutenu	tests/load/

Tests unitaires — Budget + Auth

© 2025 GateCtr

```
// tests/unit/firewall.test.ts
import { describe, it, expect, vi, beforeEach } from 'vitest';
import { checkBudget } from '@/lib/firewall';
import { prisma } from '@/lib/prisma';
import { cache } from '@/lib/redis';

vi.mock('@/lib/prisma', () => ({
  prisma: {
    budget: { findFirst: vi.fn() },
    usageLog: { aggregate: vi.fn() },
  },
}));
vi.mock('@/lib/redis', () => ({
  cache: { get: vi.fn(), set: vi.fn(), incr: vi.fn() },
}));

describe('Budget Firewall', () => {
  beforeEach(() => vi.clearAllMocks());

  it('allows when no budget configured', async () => {
    vi.mocked(prisma.budget.findFirst).mockResolvedValue(null);
    expect((await checkBudget('u1')).allowed).toBe(true);
  });

  it('blocks on hard stop + exceeded', async () => {
    vi.mocked(prisma.budget.findFirst).mockResolvedValue({
      maxTokensPerDay: 1000, hardStop: true, alertThresholdPct: 80,
    } as any);
    vi.mocked(cache.get).mockResolvedValue('1001');
    const r = await checkBudget('u1');
    expect(r.allowed).toBe(false);
    expect(r.reason).toContain('Daily token limit');
  });

  it('allows but warns at threshold (soft limit)', async () => {
    vi.mocked(prisma.budget.findFirst).mockResolvedValue({
      maxTokensPerDay: 1000, hardStop: false, alertThresholdPct: 80,
    } as any);
    vi.mocked(cache.get).mockResolvedValue('820');
    expect((await checkBudget('u1')).allowed).toBe(true);
  });
});

// tests/unit/plan-guard.test.ts
describe('Plan Guard', () => {
  it('blocks webhook creation on FREE plan', async () => {
    vi.mocked(prisma.subscription.findUnique).mockResolvedValue({
      plan: { limits: { maxWebhooks: 0 } }
    } as any);
    const r = await checkPlanLimit('u1', 'webhooks');
    expect(r.allowed).toBe(false);
    expect(r.upgrade).toBe(true);
  });
});
```

Checklist Sécurité & Audit

+	HTTPS forcé sur tous les endpoints — HSTS header via Vercel
+	Auth API Key : bcrypt hash + prefix lookup — jamais de clé en clair
+	HMAC-SHA256 sur tous les payloads webhook

© 2025 GateCtr

+	Rate limiting Redis sliding window par IP et par user
+	Validation Zod sur tous les endpoints (input sanitization)
+	Chiffrement AES-256 pour les clés LLM stockées optionnellement
+	Idempotence Stripe Events (table stripe_events)
+	Prisma parameterized queries — protection SQL injection native
+	CSP headers, X-Frame-Options, X-Content-Type-Options
+	CORS configuré — whitelist origins en production
+	Audit logs immuables sur toutes les actions critiques
+	Clerk webhook SVIX signature verification
+	Retry sécurisé avec backoff exponentiel sur webhooks
o	OWASP ZAP scan sur les endpoints publics /api/v1/*
o	Dependency scanning : npm audit + Snyk en CI
o	Docker Scout image vulnerability scanning
o	Penetration testing — OWASP Top 10 manuel
o	Secret scanning : GitHub secret scanning + git-secrets pre-commit
o	SOC 2 Type II audit préparation (pour Enterprise)

© 2025 GateCtr

14 CI/CD & DevOps Complet

GitHub Actions · Vercel · Monitoring · Sécurité production

GitHub Actions — CI

```
# .github/workflows/ci.yml
name: CI
on:
  push:    { branches: [main, develop] }
  pull_request: { branches: [main, develop] }

jobs:
  lint-typecheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v3
        with: { version: 8 }
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'pnpm' }
      - run: pnpm install --frozen-lockfile
      - run: pnpm lint && pnpm type-check

  test-unit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v3; with: { version: 8 }
      - uses: actions/setup-node@v4; with: { node-version: '20', cache: 'pnpm' }
      - run: pnpm install --frozen-lockfile
      - run: pnpm test:coverage
      - uses: actions/upload-artifact@v4
        with: { name: coverage, path: coverage/ }

  test-integration:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env: { POSTGRES_USER: gatectr, POSTGRES_PASSWORD: secret, POSTGRES_DB: gatectr_test }
        ports: ['5432:5432']
        options: --health-cmd pg_isready --health-interval 10s --health-retries 5
      redis:
        image: redis:7-alpine
        ports: ['6379:6379']
        options: --health-cmd "redis-cli ping" --health-interval 10s --health-retries 5
    env:
      DATABASE_URL: postgresql://gatectr:secret@localhost:5432/gatectr_test
      REDIS_URL: redis://localhost:6379
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v3; with: { version: 8 }
      - uses: actions/setup-node@v4; with: { node-version: '20', cache: 'pnpm' }
      - run: pnpm install --frozen-lockfile
      - run: npx prisma migrate deploy && npx prisma db seed
      - run: pnpm test:integration

  test-e2e:
    runs-on: ubuntu-latest
    needs: [lint-typecheck, test-unit]
    steps:
```


© 2025 GateCtr

```
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v3; with: { version: 8 }
      - uses: actions/setup-node@v4; with: { node-version: '20', cache: 'pnpm' }
      - run: pnpm install --frozen-lockfile
      - run: pnpm playwright install --with-deps chromium
      - run: pnpm test:e2e
      - uses: actions/upload-artifact@v4
        if: failure()
        with: { name: playwright-report, path: playwright-report/ }
```

GitHub Actions — Deploy

```
# .github/workflows/deploy.yml
name: Deploy
on:
  push: { branches: [main] }

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v3; with: { version: 8 }
      - uses: actions/setup-node@v4; with: { node-version: '20', cache: 'pnpm' }
      - run: pnpm install --frozen-lockfile

      - name: Run DB Migrations
        run: npx prisma migrate deploy
        env: { DATABASE_URL: ${{ secrets.PRODUCTION_DATABASE_URL }} }

      - name: Deploy to Vercel
        uses: amondnet/vercel-action@v25
        with:
          vercel-token: ${{ secrets.VERCEL_TOKEN }}
          vercel-org-id: ${{ secrets.VERCEL_ORG_ID }}
          vercel-project-id: ${{ secrets.VERCEL_PROJECT_ID }}
          vercel-args: '--prod'

      - name: Notify Slack on Success
        if: success()
        uses: slackapi/slack-github-action@v1
        with:
          payload: '{"text":"GateCtr deployed to production ■ — ${{ github.sha }}"}'
        env: { SLACK_WEBHOOK_URL: ${{ secrets.SLACK_DEPLOY_WEBHOOK }} }

      - name: Notify Failure
        if: failure()
        uses: slackapi/slack-github-action@v1
        with:
          payload: '{"text":"GateCtr deploy FAILED ■ — check Actions"}'
        env: { SLACK_WEBHOOK_URL: ${{ secrets.SLACK_DEPLOY_WEBHOOK }} }
```

vercel.json — Configuration production

```
{
  "framework": "nextjs",
  "buildCommand": "pnpm build",
  "installCommand": "pnpm install --frozen-lockfile",
  "regions": ["cdg1","iad1"],
  "headers": [
    {
```


© 2025 GateCtr

```
      "source": "/(.*)",
      "headers": [
        { "key": "X-Content-Type-Options",    "value": "nosniff" },
        { "key": "X-Frame-Options",           "value": "DENY" },
        { "key": "X-XSS-Protection",          "value": "1; mode=block" },
        { "key": "Strict-Transport-Security", "value": "max-age=31536000; includeSubDomains" },
        { "key": "Referrer-Policy",           "value": "strict-origin-when-cross-origin" }
      ]
    },
    {
      "source": "/api/(.*)",
      "headers": [
        { "key": "Cache-Control", "value": "no-store, no-cache" },
        { "key": "Pragma",        "value": "no-cache" }
      ]
    }
  ]
}
```

Checklist DevOps finale

+	Docker Compose dev : app + postgres + redis + worker + stripe CLI
+	Dockerfile multi-stage production (node:20-alpine, ~180MB)
+	GitHub Actions CI : lint + typecheck + unit + integration + E2E
+	Deploy pipeline : migrate → Vercel prod → Slack notify
+	Vercel regions EU (cdg1) + US (iad1) + security headers
+	Variables d'env dans Vercel + Doppler sync
+	Prisma migrations versionnées, appliquées en CI avant deploy
+	Sentry client + server + edge configuré
+	Health check /api/health (DB + Redis + Queue)
+	Rate limiting Redis sur tous les endpoints API publics
+	HMAC webhook signing + retry backoff exponentiel
+	Audit logs sur toutes actions critiques (immuables)
o	Uptime Robot : ping /api/health toutes les 5 min
o	Backup PostgreSQL daily + 30j retention
o	Axiom log aggregation + alertes
o	k6 load tests : 500 req/s cible
o	Docker Scout vulnerability scanning en CI
o	PagerDuty on-call routing incidents critiques

© 2025 GateCtr

gatectr.io · Developer Documentation v2.0 · Confidential
© 2025 GateCtr

© 2025 GateCtr